All of these libraries move away from the older-style assert methods and let you express your expectations in a more fluent and concise manner. In particular, they propose a number of higher-level assertions on collections and can be extended to work with domain objects, which helps you to avoid having to put too much logic within your unit tests.

Imagine you want to say that the member status should initially be Bronze. In traditional JUnit, you'd write something like this:

```
assertEquals(BRONZE, member.getStatus());
```

The parameter order and the somewhat clumsy wording make this sort of assertion less than ideal. It doesn't read fluently or naturally, which limits your ability to express your expectations easily and quickly.

An equivalent Hamcrest assertion, on the other hand, would look like this:

```
FrequentFlyer member = FrequentFlyer.withFrequentFlyerNumber("12345678")
                                    .named("Joe", "Bloggs");
assertThat(member.getStatus(), is(FrequentFlyerStatus.BRONZE));
```

FEST-Assert does something similar, but using a different syntactic structure:

```
assertThat(member.getStatus()).isEqualTo(FrequentFlyerStatus.BRONZE);
```

In NUnit, you could write something like this:

```
Assert.That(member.getStatus(), Is.EqualTo(FrequentFlyerStatus.BRONZE));
```

The .NET Fluent Assertions library uses a more natural style built around the word "should:"

```
Member.getStatus().Should().Be(FrequentFlyerStatus.BRONZE);
```

All of these libraries propose a rich set of matchers, including a number of convenient operations on lists. For example, to check that the list of unachieved statuses contains both Gold and Platinum, you could write the following Hamcrest assertion:

```
assertThat(member.getUnachievedStatuses(), hasItems(GOLD,PLATINUM));
```

In FEST-Assert, the equivalent would be similar:

```
assertThat(member.getUnachievedStatuses()).contains(GOLD,PLATINUM);
```

With Fluent Assertions, the assertion might look like this:

```
member.getUnachievedStatuses().Should().Contain(GOLD).And.Contain(PLATINUM);
```

All of these libraries also allow more complex expressions. For example, suppose you wanted to verify that all of the Frequent Flyers in a particular group were under the age of 18. You could express this quite elegantly in Hamcrest like this:

```
List<Integer> memberAges = ...;
assertThat(memberAges, everyItem(lessThan(18)));
```

You could also do something similar in NUnit:

```
Assert.That(memberAges, Has.All.LessThan(18));
```